61%

EIE 1st year project

Group no 8- Kung Fruit - Ravichandran, Treon, Kalanathan

by Keerthanen Ravichandran

FILE	FPGA_PROJECT-3.PDF (938.25K)		
TIME SUBMITTED	19-MAY-2017 11:09PM	WORD COUNT	4474
SUBMISSION ID	73015943	CHARACTER COUNT	24432

Imperial College London

Department of Electrical and Electronic Engineering

EIE Year 1 FPGA Project

A.Treon

K. Ravichandran

P. Kalanathan

Contents

Introduction 3

Updated description 3

Progress since Interim Report 3

Updated Schematics 4

Project Planning and Management 4

Design Process 6

Design Specification 6

Process of Design 7

Proposed Solutions 7

Final Design Selection 12

Outcomes 13

Conclusions 13

Future Improvements 14

Bibliography ..... **Error! Bookmark not defined.**

Introduction

Updated description

This final project report entails the revised expectations and outcomes of the project. It explores the methods the team employed and the lessons they learnt from trying to implement an effective hardware solution to running an image processing system on a Xilinx ZYBO FPGA board. It provides a summary of the key changes made since the interim management report, the revised design of the system and key takeaways from the process.

The interim management report revolved around designing a kinetic version of *Fruit Ninja*- a game in which the user cuts falling fruit by swiping on the touch-screen. To achieve this the team would, using an object-tracking system to track a coloured ball, replace the touch screen element with a full body interface. A camera would pick up the hand movements of the player to translate to cutting fruits.

Despite the team's initial goal to design an interactive game- *Kung Fruit*- the time taken to create an effective RGB to HSV converter proved to be longer than expected. As a result, the focus was shifted to optimisation of a program that the FPGA could run to detect a coloured ball from an image and track it. This was the core idea of the project, with the game only serving to provide an interactive context around the ball-tracking system.

Progress since Interim Report

The system will sweep through all the pixels on the screen, gathering the data for the red, green, and blue values in each cell. Through an algorithm, these values can be transformed into an HSV reading (Hue, Saturation, and Value). In the HSV space, isolating a single colour from the spectrum is vastly easier (a specific range of angles in hue translates to a colour). [3] With the colour of the ball having been isolated, the HSV values are then re-converted into RGB values and output.

To determine the dimensions of the ball, a 3x3 matrix will sweep across the screen and apply a filter to determine an edge for the isolated colour (chosen by matching it to the chosen ball colour). Thus, an outline for the ball can be created and the movement of this will be mapped as the ball moves across the screen.

The final program will then be optimised for hardware functionality using Vivado HLS to turn the C code into VHDL. [14] To increase speed, the process will be pipelined, ensuring that operation can be carried out in parallel to increase the frames/sec rate.

Updated Schematics

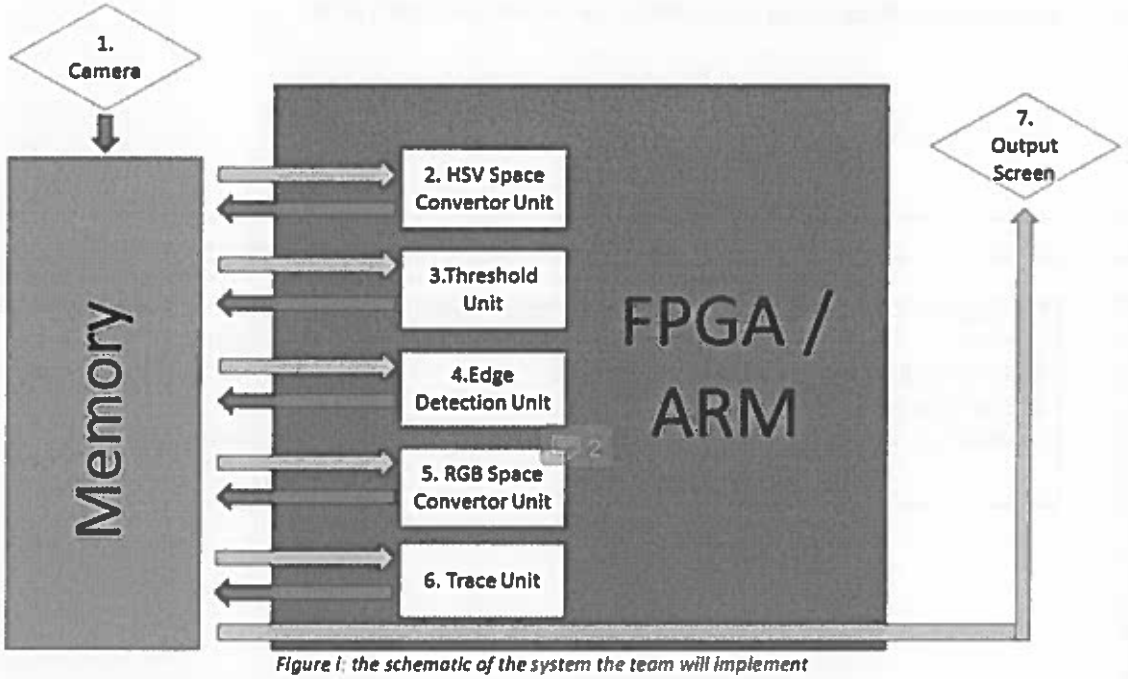


Figure i: the schematic of the system the team will implement

Project Planning and Management

The has been drastically changed since the interim report to account for changes in the final system design and the research time required to fully understand the capabilities of the hardware.

During the early research period, OpenCV was considered as an option with regards to implementing the functions that would provide an object-tracking system. [4] OpenCV (standing for Open Computer Vision) is a C++ library with tailor-made functions geared towards object tracking and computer vision.

However, after trying to implement this software in SDSoC, it was decided that *OpenCV* would not work on the hardware provided, since OpenCV requires a Linux operating system which could not be put on the ZYBO board. The *ZYBO Board* provided is limited to merely 240k fast block RAM, whereas much more memory is needed for the Linux OS. [5] The programs the team could produce using OpenCV were far more complex, however the team would have to seek an alternative route in order to implement the solution on the FPGA. Moreover, the implementation of selected C++ functions detached from the OpenCV library as a whole was beyond the capability of the team, and having consulted *Dr Bouganis*, the idea of using OpenCV functions was discarded.

The time spent on researching *OpenCV* significantly reduced the timeframe available to produce the system the team intended to design originally, driving the team to decide to work on the core hardware system of the *Kung Fruit* game- the object tracking solution- and if time was available at the end of the project, produce the game that would overlay the real-time ball-tracking system.

As several key milestones had not been met and the timeframe to complete the project had been reduced, new responsibilities were allocated as the team developed the ball-tracking system to ensure that the foundations of the game were in place. In addition, minor details- such as finding the appropriate camera gear- were not necessary as other simple solutions were found. The team held a pivot meeting which consisted of splitting the revised tasks into 3 parts.

Revised responsibilities tasks:

Armaan	Keerthanen	Prasana
Develop trace element of algorithm Research on RGB and HSV, as well as hardware implementation Construction of report	Develop motion tracking algorithm for hardware Debugging Optimisation of software code Edge detection	Producing RGB to HSV converter Converting RGB back to HSV Producing RGB colour detection

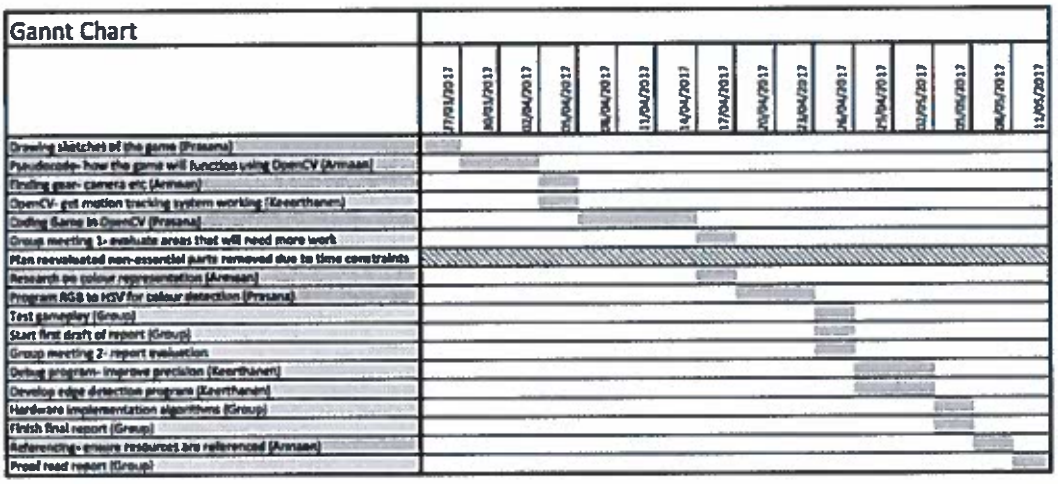


Figure ii: a Gantt chart updated to set out the revised task break-down

Design Process

Design Specification

A successful end product has to be tailored to the final users needs. The product needs to be durable and easy to use, provide a desirable user interface, and competitive in relation to other options in the market. [17] We considered the following factors:

- Performance- this was one of the most factors of the project, to provide an entertaining and captivating experience, a lot of time was spent improving the frames per second and implementing the system on hardware
- Environment- the final product would require encasing to ensure that the FPGA is not damaged, however this is not the main objective of the project
- Maintenance- the FPGA needs to be programmed once and will work, given no faults appear on the ZYBO board.
- Competition- other groups are producing similar products but there are no patents on the algorithms used.
- Shipping- the product should be shipped in a cardboard box and include silica gel to prevent damage if it shipped by sea
- Quality and reliability- the system must be very reliable and should be tested thoroughly to ensure it works
- Disposal- the ZYBO board is disposal and can be recycled as it's metal can be valuable
- Ergonomics- The ZYBO board by itself can cause cuts when touching as there are sharp metal components on the board. Therefore, a rounded plastic case should cover board. The case will have cuttings for i/o devices.
- Life in service: The device should work reliably unless any accidental falls or spillage on the board.
- Target product cost: The standalone device retails at £190. The group's code for a ball tracking is free and open-source.
- Testing: The group's code should be tested at least 50 times. The code should be checked by each member of the group. Comments should be on code so consumers should understand how the code works.
- Installation: The installation for the programme should be found on the Xilinx website regarding on ZYBO ZYNQ 7000

Process of Design

As specified in the interim report, the team met at least once a week during the Easter break to discuss progress and set goals and tasks for the next meeting. This way, setbacks were identified and averted much more effectively. Although each member had tasks to perform (which were laid out with the aid of a Gantt chart to gauge timeframes), a high degree of co-collaboration was employed to speed up the process of understanding the theory and then later to assess and improve code.

There were various stages where a decision had to be made regarding the teams plan of future action. Each decision was either broken down via a SWOT analysis, or in the case of concept generation, a brainstorm. This approach proved to be vastly effective, especially at the various pivot points in the design process.

During the process of coding the RGB to HSV converter, the team found that an iterative improvement process was most effective. Initially, two paths were pursued simultaneously- a float model and an u16 model- in order to ensure the best possible solution was implemented. [2] First the most general aspects were coded, and later any small details were implemented and any subtle changes were made. This "Zooming-in" approach made sure than any large underlying issues were addressed early in the coding process rather than too late. The code was completed centrally on one device, as the team found that separating the tasks into individual jobs was more counterproductive and in fact slowed the process down. To streamline the debugging process the values of the important variables were all printed and compared to the desired values (acquired from RGB to HSV converters on Google) in prototype code to avoid having to build and run the code on the FPGA which was time consuming.

For implementing edge detection, inspiration was drawn from other image processing techniques the team had previously encountered in background research. The sobel filter, which is a pre-set on the ZYBO, employs edge detection on a grayscale image. The image that the team would process would first be converted to a grayscale image so a sobel filter could easily detect the edge.

Proposed Solutions

The group chose to convert RGB space into HSV space due to several reasons. [7] HSV colour space is a close representation of how eyes identify colours. HSV also allows you to breakdown colour in terms of intensity of brightness. Also HSV allows the group to choose particular colour range easily using the hue in HSV. As they have to select a particular angle range to choose a set of colours.

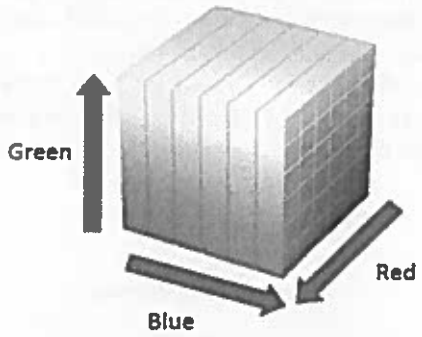


Figure iii: A pictorial representation of the RGB colour space, a 3D Cartesian system with each axis corresponding to Red, Green, and Blue. [6]

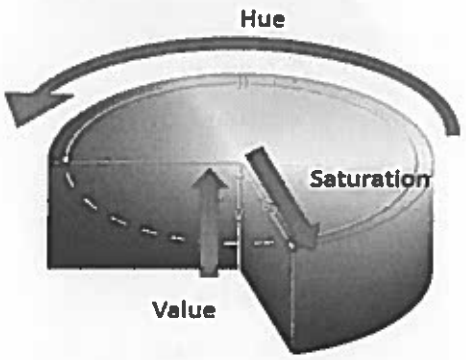


Figure iv: a pictorial representation of the HSV colour space, with Hue represented in an angular direction, saturation in a radial direction, and value in the direction of the height of the cylinder (black corresponds to 0 value) [1]

There were several facets that contributed to the complete object-tracking system. The list of proposed solutions encompasses the various methods the team considered in implementing the solution.

Float versus u16

The group set out to implement two alternate methods to get HSV space converted into RGB space. The first method was to use the data type "u16" (unsigned 16 bit binary), hence using only integers, which would allow the team to produce a faster algorithm. The second method was to use the data type "float". Float allows the use of decimal values which enables higher precision in RGB to HSV conversion. [8]

At face value, the easiest way to implement the converter would have been to use floats. As an educational exercise, the team quickly managed to build and run a RGB to HSV converter using floats on the FPGA. The results were initially positive- the accuracy was impressive and the process was easily coded and de-bugged. The key drawback from using floats was the extra data being stored (decimal values). This slowed the program down and corresponded to a significantly lower frames/sec rate than desired.

Despite being much faster to run on hardware, u16 came with several drawbacks. Decimal values weren't stored, meaning values between 0 and 1 were discarded. Also, the hue value needed to be between 0 and 360 degrees, which was not possible in u16 due to a max value of 255. To get round this issue, the algorithm was modified to incorporate an appropriately weighted angle spectrum between 0 and 255 with 6 equal segments (0-43, 43-86 etc.). However, despite the team's best attempts, a satisfactorily accurate RGB to HSV conversion was almost impossible with the data type u16.

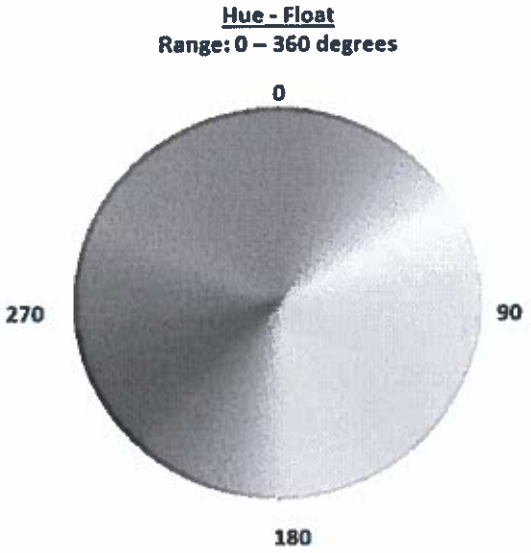


Figure v: the hue colour range spectrum, if floats are used as the data type. Floats ensured that the whole colour spectrum would be preserved in RGB to HSV

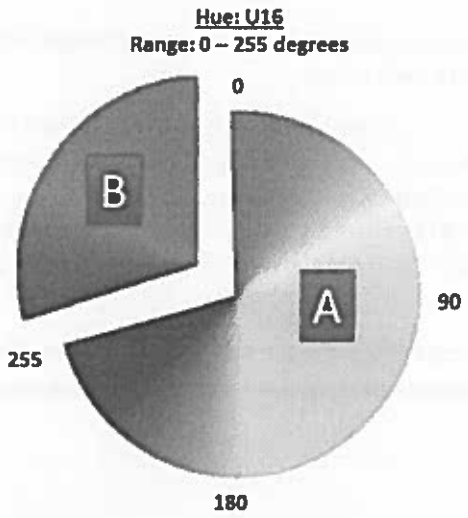


Figure vi: the hue colour range spectrum using data type u16. 0-255 degrees was successfully preserved (part A), whereas 256-360 degrees (part B) was lost/not accurately represented

Thresholding

Thresholding is the process of isolating certain colours in the spectrum. In HSV space, this task becomes vastly easier than in RGB space, due to hue being an angular value. In order to isolate a colour, the range of its hue values can be permitted and the rest can be discarded. The chosen colour of the ball to track was green (hue range 100-120 degrees). To successfully detect green, the hue values in the range would be converted to white -hue and saturation set to 0 and value set to 1- and other colour regions set to black and value set to 0.

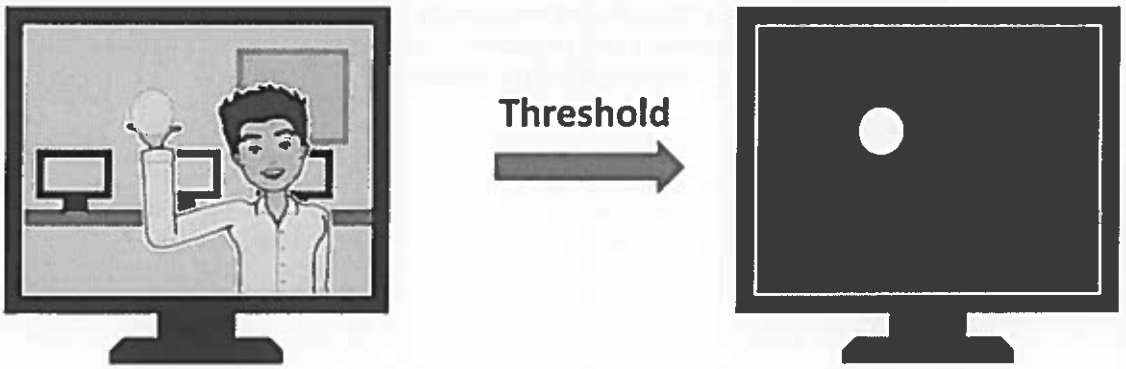


Figure vii: thresholding of the input image (left). The hue range is chosen (range needed to allow for natural variety in lighting for the image). Then, this hue range is converted to white and the rest of the colour spectrum becomes black. This can be implemented by conversions in value and saturation

Edge detection

Edge detection is using a sobel operator, which detects changes between drastic intensities between pixels in close proximity to each other.

To detect an edge using a sobel filter, first the image is converted into a grayscale image. [9] Then they apply a convolution mask on top of the desired image. The mask will be traverse through the image calculating the gradients. They use two masks to calculate the intensity of the vertical and horizontal components of the edge. Then they took the root square of the of the components to calculate the magnitude of the intensity. Also they calculate the angle of the edge using the components.

To adapt this to colour edge detection, the image is not converted to grayscale prior to applying the convolution mask. Also, the mask itself has to be modified to detect an edge of a certain hue rather than just value.

Tracking

The thresholded image is a binary image, where the white represents a 1 and the black represents 0. Hence, each pixel is either a 0 or a 1. Due to the edge detection, the program will output a vector of contour for the ball. A contour vector is used as an outline the white space which is represented by the border of the ball.[15]

The group decided to use *Hu's moments* to track the ball. *Hu's moments* will provide the coordinates of the centre of the ball. *Hu's moment* is beneficial as it does not vary with respect to rotation, translation and scale. Being invariant in scale is important as the camera will be able to track the centre of ball even when the ball moves closer to or further away from the camera, which would enlarge and diminish the 2D size of the ball detected by the camera.[16]

Finally, an infinite loop was produced, where the position of the ball is stored as "now-x" and "now-y" representing the x and y position of the ball currently respectively. These coordinates get updated into the new "now-x" and "now-y" position and the previous coordinates become "previous-x" and "previous-y" representing the previous x and y position of ball. When this happens a green trace is produced between the now and previous coordinates of the ball.

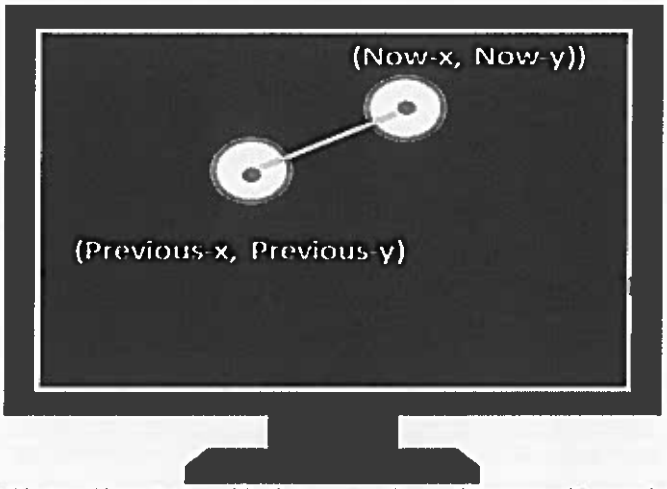


Figure viii: object tracking using transition between previous and current positions and storing the results

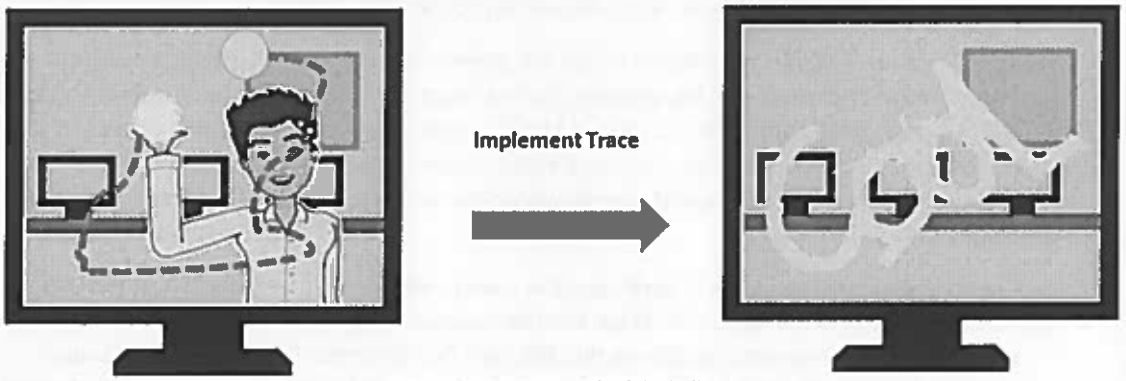


Figure ix: tracing the path of the ball

Hardware Implementation

Two adjustments had to be made to change the software code to something the FPGA could run. The team would need to consider using buffers and pipelining. The effect of implementing a buffer would ensure less overlap in the software for loops. As the matrix (sobel filter) shifts across the screen, it will read the content of pixels more than once, which is inefficient, and this can be corrected by using buffers to store the collected data. [12]Research into the implementation of buffers showed that they could contribute to a 10x increase in the speed of the program.

Final Design Selection

The team met to discuss which design was more effective for the HSV to RGB converter. From the simulations in software, the decision was clear. Although u16 could operate at a high frames/sec rate, it proved to be inaccurate in calculations due to data loss (decimal cannot be incorporated in the system). This translated to a sub-par detection algorithm. Furthermore, the scaling of the colour spectrum to 0-255 rather than 0 to 360 also had adverse effects on the performance of the RGB to HSV converter. The better option seems to be float implementation, despite the slow frames/second rate, due to an overarching accuracy in the conversion algorithm when compared with the u16 counterpart. Since the focus of the project had shifted to simple object tracking, the float method would ensure than the system would be able to detect the colour of the ball and isolate it much more accurately which corresponded to a superior overall object tracking solution.

The story was different when it came to implement the design in hardware. [13] The float data type simply used too much memory to run effectively on the FPGA. Left with u16 as the only viable option which the team had considered. After several iterations in refining and debugging the software code for the u16 RGB to HSV converter, the team managed to create a system that was satisfactory in accuracy (only the blue and violet hue range were not accurately represented). This was not at all a setback, as this would only mean that an appropriately chosen colour would be selected for the ball which would be tracked. Green was chosen due to its accurate representation in HSV.

The software code was then modified to isolate the green hues and reject all others. A simple if statement was implemented for this, allowing the hue range 100-120 degrees to be output and hues outside the range to be converted to 0 value (black). To more easily implement the sobel filtering for edge detection, the green hues were then converted to white (value 1, saturation 0) which left a grayscale image. The sobel filter could now be applied to the image and could detect to outline of the ball).

In order to implement the design in hardware, the team implemented pipelining, which allowed operations to be run in parallel by the FPGA and corresponded to a faster implementation, increasing the frames/sec rate. To achieve this, the code "#pragma HLS PIPELINE II = 1" was used after the double for loops in the software code (which would cycle through the pixels on the screen).

Although using buffers was also considered in the possible solutions, the team was not able to successfully find a solution that worked. With the pipelining successfully implemented and the impressive speed of the underlying u16 based code, the detection program was successful enough.

Outcomes

Conclusions

Due to the limited timeframe, the initially proposed game was not produced, however the FPGA was used to implement the underlying concept behind the game. The motion tracking features which were originally intended as the foundations of the project, became the focus.

The team learnt valuable lessons from struggling with time. Although initially the planning stage seems long, the numerous pivots in the process only served to demonstrate the importance of highly effective planning, especially with regards to prediction of possible branches where several options were available. Had the team done more research into the pros and cons of u16 versus float implementation, for example, it would have been a faster process to code the RGB to HSV converter and thus the team could have set about designing the game to incorporate this.

Communication also was an aspect which needed work. The effectiveness of face to face meetings between the team was tantamount. Although the use of a google drive to store collaborative documents and frequent discussions electronically, it was hard to make progress compared to meeting in the electronics and computer labs, and had this been more frequent, the team could have worked more efficiently and so could have produced a better solution.

The project was successful in several areas. converting from a RGB to a HSV colour scale using only u16. This is a major success considering that no doubles were used in the program, despite S and V being a decimal. As a compromise of using u16, there was a loss of hue between 255-360 degrees and the colour reproduction was not exact, however, this would require far less memory compared to if double or integer data types were to be used.

Furthermore, the project could accurately detect edges using an element of the Sobel filter algorithm. This would provide the perfect foundation for motion tracking, the next stage in the process.

There were design areas that fell short of expectations. The group tried some optimisation for hardware, including pipelining and attempting to use buffers to utilise the hardware to its maximum capacity. However, as a result of a shallow depth of understanding required, this was not as successful as hoped.

The number of frames per second could be significantly improved using buffers. Buffers were not used as there was not enough time to implement them and several errors were given when the group attempted to implement the buffers. Had buffers been implemented in the correct format, the code would have been expected to run at least 10 times as fast.

The project was pipelined using the statement, "#pragma AP PIPELINE II =1". This would have sped up the original code by 60 times, meaning the number of frames per second would be at least 10, however in application this proved difficult as the program did not successfully run as fast as the team had expected. The project could be improved if the faults in the pipelining stage were to be explored further in the future. Nevertheless, the pipelining did correspond to 60 times increase in the speed of the program, which was a success.

Future Improvements

Evidently, the next step in the process would be to implement the ball tracking system in the context of the game *Kung Fruit*.

A key objective would be to produce an interactive context for the motion tracking feature. This could be done if a program is written so that whenever the detected green trail (left by the path of the ball as it moves) overlaps a region that is occupied by a fruit, a point is scored. This would require the X and Y coordinate of the shape to be stored and the tracking program to be used to analyse the centre point of an object. Every time the user touches an area occupied by a shape, a point will be scored and an animation would accompany the fruit being cut. The program would have to be run in software as it would be difficult to compare values of the input as the FPGA runs sequentially.

The FPGA could be utilised more as it is the full power of the FPGA was not being used. A report indicated over 2,000,000 cycles were required due to the for-loop. Therefore, to ensure the full power of the FPGA is used the loop should be unrolled, as this will improve performance meaning more frames per second and an overall smoother performance. In addition, less space will be occupied on the FPGA so there would be room for further detailed analysis of the image to provide a more precise location of the ball.

In addition, more filters would improve the precision of the location indicated by the program. This could be done by working out the average X and Y coordinate using the of the green values from both RGB colour spectrum and HSV colour spectrum. There are other filters that could be explored. For instance, it would be useful to suppress any noise from other objects that are green. This would require implementing a mean 3x3 box function to work out the areas that are the object and to filter out other insignificant areas.

Bibliography

[1] XilinxInc, "Two techniques to hardware optimize an image processing algorithm using SDSoC," Xilinx, 3 November 2015. [Online]. Available: https://www.youtube.com/watch?v=JLfnIIwygXM. [Accessed 10 May 2017].

[2] J. Shutler, "Hu invariant set," 15 August 2002. [Online]. Available: http://homepages.inf.ed.ac.uk/rbf/CVonline/LOCAL_COPIES/SHUTLER3/node8.html. [Accessed 6 May 2017].

[3] Rapidtables, "RGB to HSV color conversion," 17 January 2017. [Online]. Available: http://www.rapidtables.com/convert/color/rgb-to-hsv.htm. [Accessed 1 May 2017].

[4] Rapidtables, 17 January 2017. [Online]. Available: http://www.rapidtables.com/convert/color/hsv-to-rgb.htm. [Accessed 1 May 2017].

[5] OpenCV team, "OpenCV," 2017. [Online]. Available: http://opencv.org/. [Accessed 1 April 2017].

[6] K. Hounslow, "Tutorial: Real-Time Object Tracking Using OpenCV," 11 March 2013. [Online]. Available: https://www.youtube.com/watch?v=bSeFrPrqZ2A. [Accessed 11 April 2017].

[7] N. B. Jensen, "Image Processing: 50 Image Filters C++," 10 December 2013. [Online]. Available: https://www.youtube.com/watch?v=zjxmZ4AODgI&feature=youtu.be. [Accessed 20 April 2017].

[8] H. Frank, "RGB Cube Show lowgamma cutout," 22 March 2010. [Online]. Available: https://commons.wikimedia.org/wiki/File:RGB_Cube_Show_lowgamma_cutout_a.png. [Accessed 27 April 2017].

[9] J. Marinacci, "Colour HSV Cylinder," 25 October 2009. [Online]. Available: https://joshondesign.com/wp-content/uploads/2009/10/colorhsbcylinder.png. [Accessed 2 May 2017].

[10] APPX4fun, "APPX4fun," 3 April 2014. [Online]. Available: http://cdn.marketplaceimages.windowsphone.com/v8/images/97f788bc-d201-4198-9886-fd97ba643088?imageType=ws_screenshot_max&rotation=90. [Accessed 22 March 2017].

[11] Digilentinc, "Digilentinc," 11 April 2016. [Online]. Available: https://reference.digilentinc.com/_media/zybo:zybo_rm.pdf. [Accessed 6 March 2017].

[12] Computerphile, "Finding the Edges (Sobel Operator) - Computerphile," 4 November 2015. [Online]. Available: https://www.youtube.com/watch?v=uihBwtPIBxM. [Accessed 20 March 2017].

[13] Xilinx, "SDSoC Environment User Guide," 20 July 2015. [Online]. Available: https://forums.xilinx.com/xlnx/attachments/xlnx/sdsoc/23/2/ug1028-sdsoc-getting-started.pdf. [Accessed 20 March 2017].

[14] Xilinx, "Vivado Design Suite Tutorial," 6 May 2014. [Online]. Available: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2014_2/ug871-vivado-high-level-synthesis-tutorial.pdf. [Accessed 13 March 2017].

[15] U. Sinha, "Tracking colored objects in OpenCV," [Online]. Available: http://aishack.in/tutorials/tracking-colored-objects-opencv/. [Accessed 13 March 2017].

[16] B. Z. T. S. Jan Flusser, "Moments and Moment Invariants in Image Analysis," 2007. [Online]. Available: http://zoi.utia.cas.cz/ICIP-2007. [Accessed 12 May 2017].

[17] M. A. Torres, "Lecture on Engineering Design Methods," in *Engineering Design Methods by N. Cross*, Wiley.

Group no 8- Kung Fruit - Ravichandran, Treon, Kalanathan

GRADEMARK REPORT

FINAL GRADE

/100

GENERAL COMMENTS

Instructor

PAGE 1

PAGE 2

Comment 1

?

PAGE 3

PAGE 4

Comment 2

This is not the best architecture for your system. All these functions require access to the main memory, which will decrease the fps. What you want is to have function 2 to read the image, and then pass it to function 3, ... and so on, and finally the data are written back to the main memory only once.

PAGE 5

PAGE 6

Comment 3

A good list of design specifications, but you need to provide a more precise description for each one.

PAGE 7

PAGE 8

PAGE 9

Comment 4

It depends on how you treat the data, You can assume that your u16 is a scaled number by 2^N, where N is the number of fractional binary bits

PAGE 10

PAGE 11

Comment 5

infinite loop?

PAGE 12

Comment 6

But this is exactly what we have in the demo HW function

PAGE 13

Comment 7

Any comment on the hardware resources? Any images on what has been achieved?

PAGE 14

Comment 8

? What do you mean by this? Why using buffers would increase the fps?

PAGE 15

PAGE 16

PAGE 17